FRONTEND ROADMAP

DAY 9

TypeScript Fundamentals II

Model real data with types: objects, interfaces, unions and generics — the exact skills you'll use to type React components on Day 10.

WHAT WE COVER

Objects | Interfaces | type vs interface | Unions & narrowing | Generics | Convert a project

Yesterday you learned what TypeScript is and how to type simple values and functions. Today we level up to describing **real data** — the shape of a user, a task, a product — using interfaces and type aliases, then make code reusable with generics. We finish by converting the Day 7 terminal calculator into TypeScript. These are the very tools you'll reach for when typing React props tomorrow.

TypeScript Fundamentals II

Apps are mostly about **data with a shape**: a user has a name and an email; a task has a title and a status. TypeScript lets you describe those shapes once and reuse them everywhere, so the compiler guarantees your objects always match. That single idea powers safe React components.

CLASS AGENDA · ~2 HOURS

0:00	Recap + typing objects and interfaces	§1–2
0:20	type vs interface, optional & readonly	§3–4
0:40	Arrays of objects; unions & narrowing	§5–6
1:05	Enums vs literals, then generics	§7–8
1:30	Typing callbacks; convert the calculator	§9–10
1:55	Model a Task & wrap-up	§11

ON THIS HANDOUT

PART 1 · Shaping Data

01 Typing objects

Most values in an app are objects. You can describe an object's shape inline with types on each property:

```ts
const user: { name: string; age: number } = {
  name: "Abhishek",
  age: 25,
};

user.age = 26;       // fine
user.age = "old";    // Error: 'string' is not assignable to 'number'
user.email = "...";  // Error: 'email' does not exist on this shape
```

Writing the shape inline every time gets repetitive. That's exactly what **interfaces** and **type aliases** solve.

02 Interfaces

An **interface** gives an object shape a reusable name. Define it once, use it anywhere:

```ts
interface User {
  name: string;
  age: number;
}

const a: User = { name: "Abhi", age: 25 };
const b: User = { name: "Sam", age: 30 };

function birthday(u: User): void {
  u.age = u.age + 1;
}
```

- Declare the shape once; reuse it for variables, parameters and return types.
- If an object is missing a property or has the wrong type, TS tells you immediately.
- Interface names use PascalCase by convention (`User`, `Task`).

03 type vs interface

A **type alias** can also name a shape — and it can name anything else too (unions, primitives). For objects, the two look almost identical:

```ts
interface User { name: string; age: number; }

type UserT = { name: string; age: number; };   // same idea, different keyword

// type can also name things interfaces can't:
type ID = string | number;
type Status = "todo" | "inprogress" | "done";
```

- Use an **interface** for object shapes — it's the common convention, especially for React props.
- Use a **type** when you need unions, literals, or to name a primitive.

- Day-to-day, either works for objects; pick one and stay consistent.

04 Optional & readonly properties

Real data isn't always complete. Mark properties that may be missing with `?`, and lock ones that shouldn't change with `readonly`.

```ts
interface User {
  id: number;
  name: string;
  email?: string;        // optional -- may be undefined
  readonly role: string; // set once, can't be reassigned
}

const u: User = { id: 1, name: "Abhi", role: "admin" };  // email omitted: OK
u.role = "user";   // Error: cannot assign to 'role' because it is read-only
```

- `email?` means the property might not be there — TS makes you handle the undefined case.
- `readonly` protects values that should never change after creation (like an id).

05 Arrays of objects

The most common real-world shape: a list of items. Combine an interface with the array type from Day 8.

```ts
type Status = "todo" | "inprogress" | "done";

interface Task {
  id: number;
  title: string;
  status: Status;
}

const tasks: Task[] = [
  { id: 1, title: "Learn TypeScript", status: "todo" },
  { id: 2, title: "Build an app",     status: "inprogress" },
];

const todo = tasks.filter((t) => t.status === "todo");   // t is fully typed
console.log(todo.length);
```

Notice how `t` inside `filter` already knows it's a `Task` — so `t.status` autocompletes and only accepts valid statuses. This is the payoff.

> **TRY IT YOURSELF**
>
> Create an `interface Product { id: number; name: string; price: number; }` and an array of three products, then log the ones under 100.

PART 2 · More Power

06 Unions & narrowing

A union value could be one of several types, so TypeScript makes you **narrow** it — prove which one it is — before using type-specific features.

```ts
function printId(id: string | number): void {
  if (typeof id === "string") {
    console.log(id.toUpperCase());   // TS knows it's a string here
  } else {
    console.log(id.toFixed(0));      // and a number here
  }
}
```

- A `typeof` check (or an `if`) narrows the union inside each branch.
- Inside a branch, TS lets you safely use methods for that specific type.

07 Enums vs literal unions

When a value should be one of a fixed set of options, you have two tools. A **literal union** is the lightweight favourite:

```ts
type Status = "todo" | "inprogress" | "done";   // literal union
let s: Status = "todo";

// an enum is an alternative, more formal option:
enum Role { Admin, Editor, Viewer }
let r: Role = Role.Admin;
```

- Literal unions are simple, readable, and disappear at compile time — usually preferred.
- Enums create a real object at runtime; handy occasionally, but reach for unions first.

08 Generics: reusable typed code

Sometimes you want a function or container that works with **any** type but still stays type-safe. That's what **generics** do — a type “variable”, written in angle brackets, filled in when you use it.

```ts
function firstItem<T>(items: T[]): T {
  return items[0];
}

const n = firstItem<number>([10, 20, 30]);   // n is a number
const s = firstItem(["a", "b"]);             // T inferred as string

// You've already seen the built-in generic Array<T>:
let scores: Array<number> = [90, 85];        // same as number[]
```

You don't have to write many generics yourself yet — but recognising them matters, because React's `useState` is generic: `useState<number>(0)` tells React the state is a number. Today's the day that clicks.

09 Typing functions & callbacks

Functions can be values passed to other functions (callbacks) — and those can be typed too. This is exactly how you'll type event handlers in React.

```ts
// a variable holding a function: (n: number) => number
const double: (n: number) => number = (n) => n * 2;

// a callback parameter, fully typed
function repeat(times: number, action: (i: number) => void): void {
  for (let i = 0; i < times; i++) action(i);
}

repeat(3, (i) => console.log("step", i));
```

- The type `(n: number) => number` describes a function's inputs and output.
- Typed callbacks mean TS checks you use them correctly — no surprises at runtime.

PART 3 · Apply It

10 Project: convert the calculator

Let's bring Day 7 and today together by adding types to the terminal calculator. First, model the operator as a literal union, then type the function fully.

```ts
type Operator = "+" | "-" | "*" | "/";

function calculate(a: number, op: Operator, b: number): number | string {
  switch (op) {
    case "+": return a + b;
    case "-": return a - b;
    case "*": return a * b;
    case "/": return b !== 0 ? a / b : "Error: cannot divide by zero";
  }
}

console.log(calculate(8, "+", 5));    // 13
console.log(calculate(10, "/", 0));   // Error: cannot divide by zero
calculate(8, "%", 5);                 // Error: "%" is not a valid Operator
```

- The `Operator` union means a typo like `%` is caught by the compiler, not by a user.
- The return type `number | string` honestly describes both outcomes (a result or an error message).

Typing the input helper

The readline helper from Day 7 gets types too — it takes a string question and resolves to a string answer:

```ts
import * as readline from "readline";

const rl = readline.createInterface({ input: process.stdin, output: process.stdout });
const ask = (question: string): Promise<string> =>
  new Promise((resolve) => rl.question(question, resolve));
```

> **TRY IT YOURSELF**
>
> Add a power case (`"**"`) to the `Operator` union and the `switch` . Notice TS reminds you to handle the new case.

11 Model preview: a typed Task

Here's a taste of tomorrow. This `Task` interface is **exactly** the kind of type you'll use to describe a React component's props — so a component always receives the right data.

```ts
interface Task {
  id: number;
  title: string;
  status: "todo" | "inprogress" | "done";
  done?: boolean;
}

// Tomorrow, a React component will say: function TaskCard(props: Task) { ... }
// and TypeScript guarantees every TaskCard gets a valid task.
```

If you're comfortable writing and reading interfaces like this, you're ready for React with TypeScript.

Practice Challenges

Pick a few to cement today's ideas:

- Define an `interface Book { title: string; author: string; year: number; inStock?: boolean; }` and make an array of books.
- Write a generic `lastItem<T>(items: T[]): T` and test it with numbers and strings.
- Model a `type Theme = "light" | "dark"` and a function that returns a background colour for each.
- Fully type the Day 7 guessing game or word-counter project.

Conclusion & what's next

You can now describe real data with **interfaces** and **type aliases**, handle optional and readonly fields, narrow **unions**, and write reusable **generic** code. You also converted a real project to TypeScript end to end.

Next class (Day 10): React. We'll see why manual DOM work doesn't scale, set up a project with **Vite + TypeScript**, build components, and make them interactive with **useState** — typing everything with the interfaces you just mastered.

> QUICK RECAP
>
> Describe data shapes with **interfaces** / **type** aliases · **?** = optional, **readonly** = locked · type **arrays of objects** for lists · **narrow** unions before use · **generics** (like `useState<T>`) keep reusable code type-safe.